

OBJECT ORIENTED SYSTEMS (OOS) — 2023

QUESTION 2 COMPLETE DETAILED SOLUTION

Introduction

Question 2 of the 2023 Object Oriented Systems examination belongs to CO2 and focuses on some of the most important practical concepts of Java programming:

-  Multithreading
-  Runnable Interface
-  Thread Lifecycle
-  Static Context
-  `sleep()`, `wait()`, `join()`, `notify()`
-  File Handling
-  `BufferedReader`
-  Command Line Arguments

These topics are extremely important because modern software systems heavily depend upon:

- concurrent execution,
- file processing,
- synchronization,
- and efficient input-output management.

Real-world applications such as web servers, operating systems, Android applications, multiplayer games, and enterprise backend systems all rely extensively on multithreading and Java I/O  .

This question is highly conceptual and tests both theoretical understanding and program analysis capability.

QUESTION 2(a)(i)

 **Complete the statement: The Java uses threads to enable the entire environment to be _____**

Options: A) Symmetric B) Asymmetric C) Synchronous D) Asynchronous

 Answer

 **D) Asynchronous**

Detailed Explanation

Java uses multithreading to support:  **Asynchronous Execution**

Asynchronous execution means:

- multiple operations can proceed independently,
- tasks execute concurrently,
- one task does not block the entire program.

Without multithreading:

- application responsiveness decreases,
- CPU utilization becomes poor,
- background operations become impossible.

🌟 Real-World Example

Suppose a browser is downloading files, playing video, loading a webpage, and handling user clicks simultaneously. This becomes possible because of asynchronous multithreaded execution 🚀.

🎯 Conclusion

Java threads enable asynchronous execution, improving responsiveness and efficient resource utilization.

📄 QUESTION 2(a)(ii)

❓ Which of the following constructor of class Thread is valid?

Options: A) `Thread(Runnable threadOb, int priority)` B) `Thread(int priority)` C) `Thread(Runnable threadOb, String threadName)` D) `Thread(String threadName, int priority)`

✅ Answer

✅ **C) `Thread(Runnable threadOb, String threadName)`**

🌟 Detailed Explanation

The Java `Thread` class provides several overloaded constructors. Among the given options, only `Thread(Runnable threadOb, String threadName)` is valid.

🌟 Why Other Options are Invalid?

Java `Thread` constructors do NOT directly accept priority values as constructor parameters. Thread priority is set separately using the `setPriority()` method.

✅ Correct Example

```
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Thread Running");  
    }  
}  
  
class Main {  
    public static void main(String args[]) {  
        Thread t = new Thread(new MyTask(), "WorkerThread");  
        t.start();  
    }  
}
```

Output

```
Thread Running
```

Conclusion

Java `Thread` constructors support `Runnable` objects and thread naming but not direct priority initialization.

QUESTION 2(a)(iii)

 **Analyze the following code and choose the correct option.**

```
public abstract class Test implements Runnable {  
    public void doSomething();  
}
```

Options: A) Program will not compile because `run()` not implemented B) Program will not compile because abstract methods absent C) Program compiles fine D) None of the above

 Answer

 **C) Program compiles fine**

Detailed Explanation

This class is declared `abstract`. Thus it is NOT required to implement `run()` immediately. Abstract classes may contain abstract methods and incomplete implementations. Hence the compiler allows it.

Important Concept

If the class were NOT abstract (`class Test implements Runnable`), then the `run()` implementation becomes compulsory. Otherwise, a compiler error occurs .

🌟 Correct Runnable Implementation

```
class Demo implements Runnable {  
    public void run() {  
        System.out.println("Running");  
    }  
}
```

🎯 Conclusion

Abstract classes implementing interfaces are not forced to implement all interface methods immediately 🚀.

📄 QUESTION 2(a)(iv)

? Analyze the following code and choose the correct option.

```
public class Test implements Runnable {  
    public void run() {  
        System.out.print("Successful ");  
    }  
  
    public static void main(String[] args) throws InterruptedException {  
        Thread thread1 = new Thread(new Test());  
        thread1.start();  
        thread1.start();  
        System.out.println(thread1.getState());  
    }  
}
```

✅ Answer

✅ Runtime Error

🌟 Detailed Explanation

A Java thread can be started ONLY ONCE ⚠️. Initially, `thread1.start();` creates and starts the thread successfully. However, the second call `thread1.start();` attempts restarting the same thread. Java does NOT allow restarting terminated/already-started threads.

Thus JVM throws: 🔥 **IllegalThreadStateException** during runtime.

🌟 Important Thread Lifecycle Concept

Thread states include: **NEW**, **RUNNABLE**, **RUNNING**, **WAITING**, **TERMINATED**. Once a thread leaves the **NEW** state through `start()`, it cannot return back.

🎯 Conclusion

A thread object can be started only once in Java; otherwise a runtime exception occurs 🚀.

QUESTION 2(a)(v)

🔍 **Analyze the following code and choose the correct option.**

```
public abstract class Test implements Runnable {  
    public static void main(String[] args) {  
        Thread t = new Thread(this);  
        t.start();  
    }  
  
    public void run() {  
        System.out.println("test");  
    }  
}
```

✅ Answer

✅ **A) The program does not compile because this cannot be referenced in static method.**

🔦 Detailed Explanation

The keyword `this` represents the current object reference. However, static methods belong to the class itself, not objects. Thus inside `public static void main()`, there is NO current object. Therefore, `new Thread(this)` is illegal. The compiler generates an error ⚠️.

🔦 Correct Approach

Create the object explicitly.

✅ Correct Program

```
class Test implements Runnable {  
    public void run() {  
        System.out.println("test");  
    }  
  
    public static void main(String args[]) {  
        Test obj = new Test();  
        Thread t = new Thread(obj);  
        t.start();  
    }  
}
```

Output

test

Conclusion

The keyword `this` cannot be used inside static methods because static methods have no object context .

QUESTION 2(a)(vi)

 **Analyze the following code.**

```
public class Test extends Thread {  
    public static void main(String[] args) {  
        Test t = new Test();  
        t.run();  
        t.start();  
    }  
  
    public void run() {  
        System.out.print("run-test ");  
    }  
}
```

Answer

run-test run-test

Detailed Explanation

This question tests understanding of the difference between `run()` and `start()`.

 **First Call** `t.run()`; directly invokes the method like an ordinary function call. Thus `run-test` prints. No new thread is created.

 **Second Call** `t.start()`; creates a NEW thread internally. The JVM automatically invokes `run()` again. Thus the second `run-test` prints.

Final output: `run-test run-test`

Important Difference

Method	Meaning
<code>run()</code>	Normal method call. Executes in the same thread (No multithreading).
<code>start()</code>	Creates a new thread. Actual threading (Executes concurrently).

Conclusion

Calling `run()` directly does not create a thread, whereas `start()` initiates multithreaded execution 🚀.

📄 QUESTION 2(b)

? **Indicate functionalities of `sleep()` and `wait()` along with overloaded versions.**

? **What are utilities of `join()` and `notify()` methods?**

✅ Answer

Thread coordination is one of the most important aspects of concurrent programming 🧠. Java provides several methods for pausing threads, synchronization, communication, and coordination.

💡 `sleep()`

The `sleep()` method belongs to the `Thread` class. It temporarily pauses current thread execution. ✅ **Syntax:** `Thread.sleep(1000);` (Pauses thread for 1000 milliseconds.) 💡 **Overloaded Versions:**

- `sleep(long millis)`
- `sleep(long millis, int nanos)`

💡 `wait()`

The `wait()` method belongs to the `Object` class. It causes the current thread to wait until notified. It must be used inside a synchronized block or synchronized method. 💡 **Overloaded Versions:**

- `wait()`
- `wait(long millis)`
- `wait(long millis, int nanos)`

💡 Difference Between `sleep()` and `wait()`

Feature	<code>sleep()</code>	<code>wait()</code>
Class	<code>Thread</code>	<code>Object</code>
Lock Released	No	Yes
Synchronization Needed	No	Yes

💡 `join()`

The `join()` method forces one thread to wait until another thread completes. Example: The main thread waits for a worker thread.

💡 `notify()`

The `notify()` method wakes one waiting thread. Used for inter-thread communication.

🎯 Conclusion

`sleep()`, `wait()`, `join()`, and `notify()` are fundamental thread coordination methods enabling synchronization and controlled multithreaded execution 🚀.

QUESTION 2(c)

? **Write a Java class that opens a text file, selects words containing at most one vowel, and writes them into another file using command line arguments.**

✓ Answer

This question combines file handling, string processing, vowel counting, and command-line arguments.

✓ Java Program

```
import java.io.*;

class FileDemo {

    static int countVowels(String word) {
        int count = 0;
        word = word.toLowerCase();

        for(int i = 0; i < word.length(); i++) {
            char ch = word.charAt(i);
            if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u') {
                count++;
            }
        }
        return count;
    }

    public static void main(String args[]) throws Exception {
        BufferedReader br = new BufferedReader(new FileReader(args[0]));
        PrintWriter pw = new PrintWriter(new FileWriter(args[1]));

        String line;

        while((line = br.readLine()) != null) {
            String words[] = line.split("\\s+");

            for(String word : words) {
                if(countVowels(word) <= 1) {
                    pw.println(word);
                }
            }
        }

        br.close();
        pw.close();

        System.out.println("Done");
    }
}
```

```
}  
}
```

Example Input File

```
sky  
apple  
gym  
book
```

Output File

```
sky  
gym
```

Deep Explanation

Words like **sky** and **gym** contain at most one vowel. Thus, they are written into the destination file.

Conclusion

Java file handling combined with string processing enables efficient text filtering operations .

QUESTION 2(d)

? **Write a Java class Student having: name, roll, stream, cgpa.**

? **Find name and roll of student having highest CGPA in IT stream among 400 students using `BufferedReader`.**

Answer

This problem combines object-oriented programming, `BufferedReader`, conditional searching, and object arrays.

Java Program

```
import java.io.*;  
  
class Student {  
    String name;  
    int roll;  
    String stream;  
    double cgpa;  
  
    void input(BufferedReader br) throws Exception {
```

```
        name = br.readLine();
        roll = Integer.parseInt(br.readLine());
        stream = br.readLine();
        cgpa = Double.parseDouble(br.readLine());
    }
}

class Main {
    public static void main(String args[]) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        Student s[] = new Student[400];

        double max = -1;
        String topName = "";
        int topRoll = 0;

        for(int i = 0; i < 400; i++) {
            s[i] = new Student();
            s[i].input(br);

            if(s[i].stream.equals("IT") && s[i].cgpa > max) {
                max = s[i].cgpa;
                topName = s[i].name;
                topRoll = s[i].roll;
            }
        }

        System.out.println(topName);
        System.out.println(topRoll);
    }
}
```

🔍 Deep Explanation

The program stores 400 `Student` objects. It then searches for the highest `CGPA` within the `IT` stream only. `BufferedReader` enables efficient text input.

🎯 Conclusion

This program demonstrates real-world object handling, searching, and buffered input processing in Java 🚀.